

Color Recognition Sorting Broadcast

Note: Due to unavoidable servo mechanical errors and the influence of servo center position, if the grabbing and placing positions are not ideal, you can try recalibrating the robotic arm or modify the grabbing and placing positions according to your needs.

1. Function Description

The voice module broadcasts the current block color and the robotic arm grabs and places it in the set position.

2. Startup and Operation

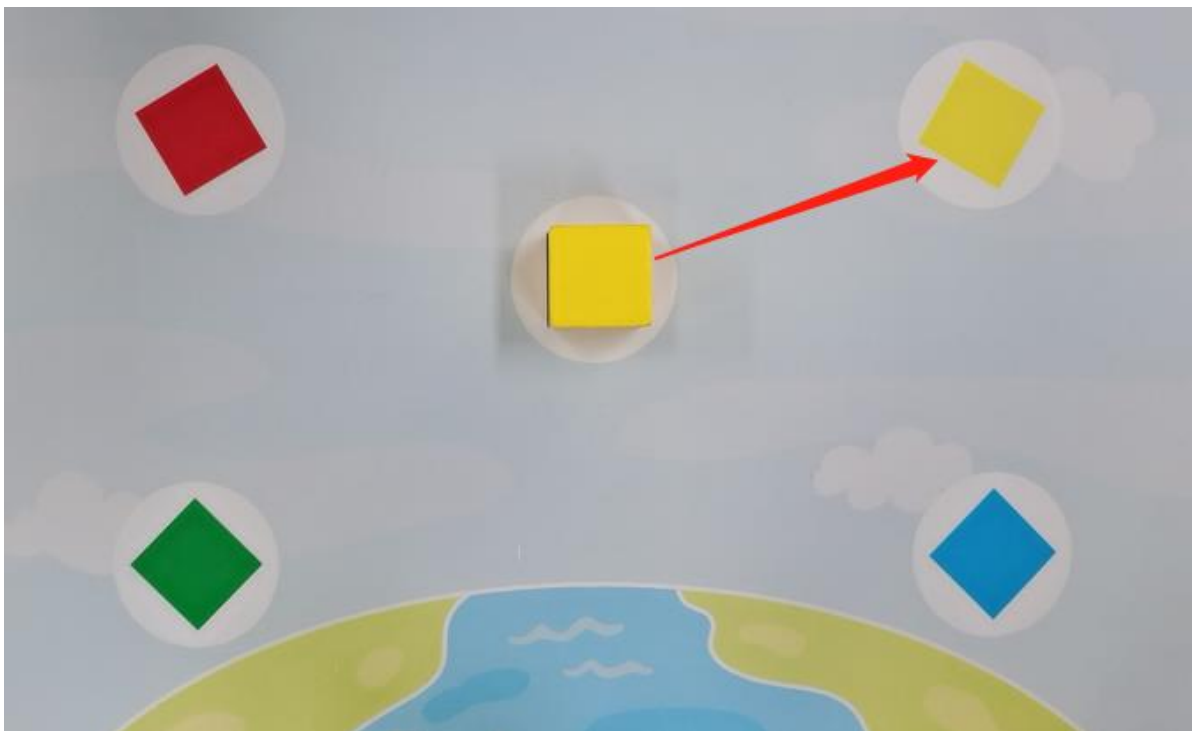
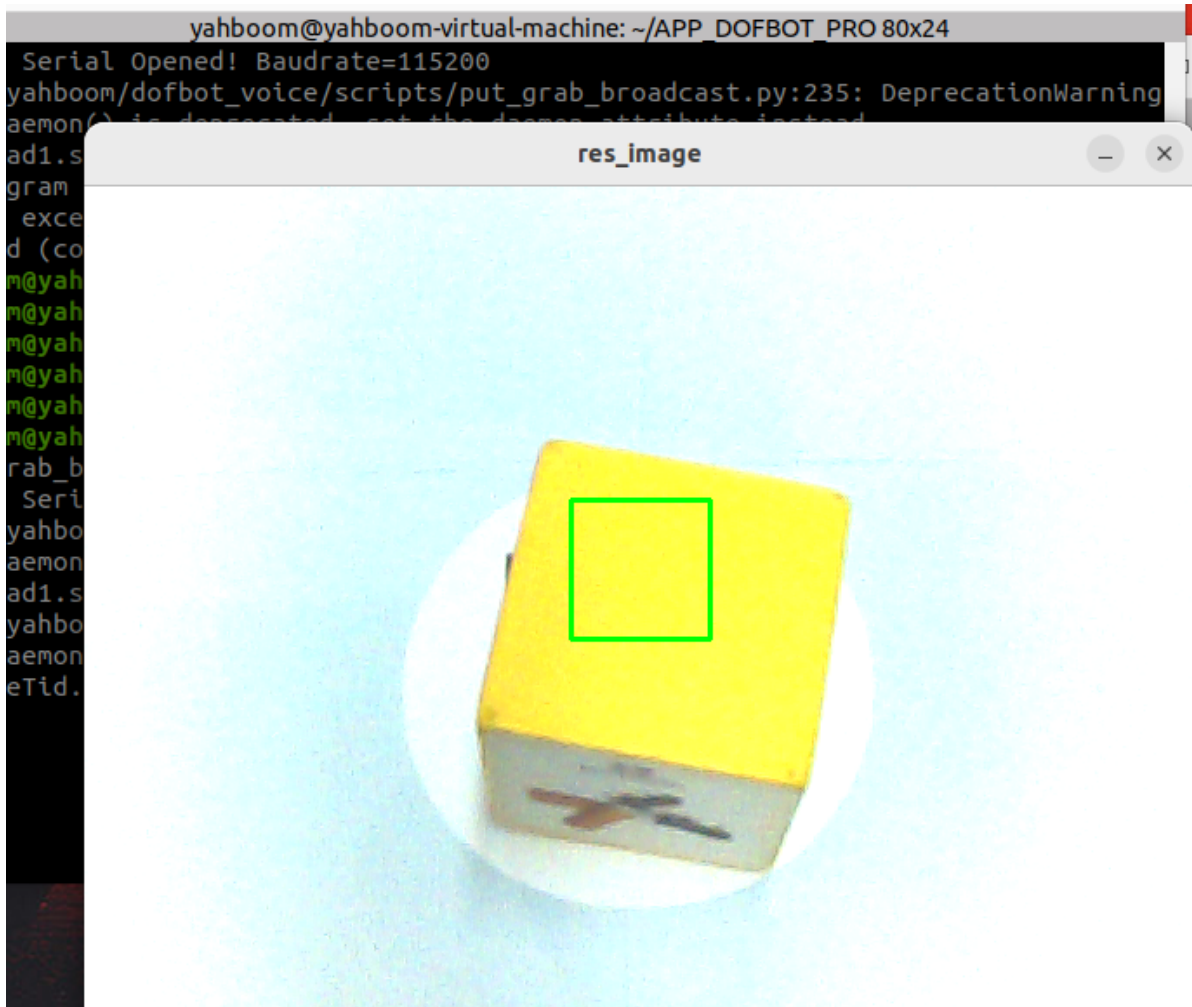
2.1 Startup

Open the mainboard terminal and enter the following command:

```
python3 ~/dofbot_voice/scripts/put_grab_broadcast.py
```

2.2 Operation Steps

After the program starts, the robotic arm automatically moves to the recognition position. Place the block in the middle gray-black position. After the robotic arm correctly recognizes 10 times continuously, the voice module will broadcast the recognized color, and the terminal will print the recognized color. Then it will automatically grab and place the block in the corresponding color grid, and finally broadcast "Placement complete". Taking the figure below as an example, the voice module will broadcast "I see Yellow" and then grab and place it in the corresponding color grid.



3. Core Code Analysis

Source code path: ~/dofbot_voice/scripts/put_grab_broadcast.py

```
# Import voice broadcast block sorting library
from speech_color_sorting import speech_color_sorting
```

```

# Create instance
sorting = speech_color_sorting()
# Call function, pass image and color hsv threshold for block color recognition
img = sorting.Sorting_grap(img, color_hsv)

# Sorting_grap function is in speech_color_sorting library
def Sorting_grap(self, img, color_hsv):
    # Canonical input image size
    self.image = cv.resize(img, (640, 480))
    # Get recognition result
    msg = {}
    if self.status == 'waiting':
        # Iterate through color channels to get recognizable results
        for key, value in color_hsv.items():
            point = self.get_Sqaure(key, value)
            if point != None: msg["name"] = key
        if len(msg) == 1:
            self.num += 1
            # whenever 10 consecutive recognitions are made and the motion
            status is waiting, execute the grabbing task
            if self.num % 10 == 0 and self.status == 'waiting':

                self.status = "Runing"
                self.arm.Arm_Buzzer_On(1)
                sleep(0.5)
                # Start grabbing thread
                threading.Thread(target=self.sorting_run, args=
(msg['name'],)).start()
                self.num = 0
        return self.image
# sorting_run function is in speech_color_sorting library
def sorting_run(self, name):
    # According to the value of name, broadcast "Recognized what color" and then
    execute self.sorting_move, passing in the placement position
    if name == "red" :
        mySpeech.void_write(88)
        time.sleep(1.0)
        mySpeech.void_write(46)
        # print("red")
        self.sorting_move(self.robot.P_RED)
        mySpeech.void_write(81)
        # Grabbing completed
        self.status = 'waiting'
    if name == "blue":
        mySpeech.void_write(88)
        time.sleep(1.0)
        mySpeech.void_write(48)
        # print("blue")
        self.sorting_move(self.robot.P_BLUE)
        mySpeech.void_write(81)
        # Grabbing completed
        self.status = 'waiting'
    if name == "green" :
        mySpeech.void_write(88)
        time.sleep(1.0)
        mySpeech.void_write(49)
        # print("green")
        self.sorting_move(self.robot.P_GREEN)

```

```
mySpeech.void_write(81)
# Grabbing completed
self.status = 'waiting'
if name == "yellow" :
    mySpeech.void_write(88)
    time.sleep(1.0)
    mySpeech.void_write(47)
    # print("yellow")
    self.sorting_move(self.robot.P_YELLOW)
    mySpeech.void_write(81)
    # Grabbing completed
    self.status = 'waiting'
```